

QA3

核心主題：多型 (Polymorphism) 與物件導向設計

一、老師再三強調的核心觀念：什麼是多型？

老師在課程一開始與最後都特別強調，許多人對「多型」有嚴重的誤解：

- **錯誤認知**：很多人（包含許多研究生與業界人士）以為程式碼裡面有「繼承 (Inheritance)」，或者使用了同名但參數不同的「多載 (Overloading)」，就代表具備了物件導向或多型。**老師強調這是大錯特錯的！** Overloading 只是同一個 function name 有不同的 signature（回傳型態或參數），這在 C 語言中就能區分，與多型毫無關聯。
- **正確定義**：多型是指當你使用**基底類別 (Base class) 的指標 (pointer) 或參考 (reference)** 來操作不同子類別 (subtype) 的物件時，程式能依據其真實的子類別，執行相對應的不同行為。
- **沒有多型的繼承毫無意義**：老師指出，如果你的程式碼只有繼承，卻沒有利用多型把「會變動的部分」與「不變的核心」隔離開來，那你的程式根本稱不上是物件導向，且未來擴充時必定需要大改程式碼。一個真正的 OOP 系統目標是達到高度的可重用性與可維護性。一個常見的錯誤範例是寫出 `Something s = new Something();` 這種程式碼，沒有將指標指派給 Base class，代表完全沒有用到多型。

二、如何實作多型：虛擬函式 (Virtual Functions)

要讓多型在 C++ 中生效，必須依賴 `virtual` 關鍵字與指標/參考。

1. **Virtual 關鍵字**：任何想要被子類別覆寫 (override) 的 method 都應該在基底類別中宣告為 `virtual`。
2. **指標與參考**：多型只對宣告為 `virtual` 的函式，且透過指標 (pointer) 或參考 (reference) 呼叫時才有效。
3. **Virtual Destructor (虛擬解構子)**：如果一個 class 有任何 method 被宣告為 virtual，那它的解構子 (destructor) 也應該宣告為 virtual，以確保未來刪除指標時，能正確呼叫到子類別的解構子並做適當的記憶體清除。
4. **為什麼不把所有函式都宣告為 Virtual？** 雖然 Java 預設所有 method 都是 virtual，但在 C++ 中，宣告 virtual 會產生額外的系統負擔 (overhead)，因為編譯器需要建立虛擬函式表 (vtable) 並維護這些機制。

三、物件切割 (Object Slicing) 與 參數傳遞

老師透過老鼠 (`Rat`) 與寵物 (`Pet`) 的繼承例子解釋這點：

- `Rat` 繼承自 `Pet`，因此 `Rat` 在記憶體中的體積比較大（前半部是 `Pet` 的記憶體，後半部是 `Rat` 特有的記憶體）。
- **Pass by Value (傳值)**：如果你直接傳遞物件（例如把 `Rat` 物件當作參數傳入吃 `Pet` 的函式），編譯器會做複製 (copy)，而因為 `Rat` 的體積大於 `Pet`，放不進去，編譯器會把 `Rat` 特有的部分「砍掉」，只複製 `Pet` 的部分，這就叫 **Object Slicing (物件切割)**。此時呼叫函式只會得到基底類別的行為，**多型將會失效**。
- **Pass by Address / Reference (傳址/傳參考)**：為了讓多型生效，應該傳遞指標 (Pointer, pass by address) 或參考 (Reference, pass by reference)。傳參考是 C++ 特有的設計，讓你用起來像在操作一般變數（不需使用 `>` 解指標），但骨子裡仍是透過記憶體位址來操作，因此能保留原始子類別的特性，成功展現多型。

四、多型的底層機制：動態連結 (Dynamic Binding)

老師解釋了編譯器是如何在幕後實作多型的：

1. **早期作法 (Function Pointer)**：在沒有 OOP 的 C 語言時代，要做到類似多型的動態呼叫，需利用函數指標 (Function pointer，例如 `void (*Move)();`)，在執行時動態把特定的函式位址塞入結構中。
2. **靜態連結 (Static Binding)**：一般的 C 程式，編譯器或連結器在編譯期間 (Compilation time) 或連結期間 (Linking time) 就已經決定好函式的記憶體位址 (logical address)。
3. **動態連結 (Dynamic Binding) 與 vtable**：
 - 只要 class 裡有宣告 `virtual` 函式，C++ 編譯器就會偷偷幫這個 class 準備一個**虛擬函式表 (Virtual Function Table, vtable)**，裡面記錄了該 class 所有 virtual function 的真實記憶體位址。
 - 該 class 的每一個物件實例 (instance)，其記憶體中都會多出一個隱藏的指標叫做 **vptr (Virtual Pointer)**，指向它所屬 class 的 vtable。
 - **實際運作**：當你寫出 `p->draw();` 時，編譯器並不知道 `p` 指向的究竟是 Base 還是 Subclass，所以它翻譯出來的組合語言指令一模一樣。它會產生一個「間接呼叫 (indirect call)」，去物件的 `vptr` 找到 vtable，再從裡面抽出正確的函數位址來執行。因此，物件內容不同，呼叫到的函式就會不同，這就是多型底層的真正實作原理。

五、抽象類別 (Abstract Class) 與建構子初始列

- **抽象類別**：如果一個 class 裡有「純虛擬函式 (Pure virtual function)」(例如宣告為 `virtual void p() = 0;`)，這個 class 就是 Abstract class。抽象類別不能被實體化 (instantiate)，你不能 `new` 它出物件。
- **成員初始化 (Member Initialization)**：當建立子類別的物件時，編譯器會先去呼叫基底類別的建構子 (Constructor)。若想傳參數給基底類別的建構子，C++ 發明了在大括號前使用冒號 `:` 的寫法，在進入子類別建構子主體之前，先把參數傳遞給基底類別建構子。

六、課程作業詳解 (HW1 ~ HW6)

HW1：虛擬解構子與多型的執行順序

- **題目內容**：有一段程式碼中 `Base* base = new A();`，呼叫 `base->baseMethod();` 後再 `delete base;`。
- **輸出結果**：

```
from Afrom Afrom Base
```
- **細節解析**：
 1. `baseMethod()` 呼叫虛擬函式 `method()`，因為實際物件是 `A`，所以印出第一個 `from A`。
 2. `delete base` 觸發解構。由於 `Base` 的解構子是 `virtual`，會先呼叫子類別 `-A()`，裡面執行 `method()`，印出第二個 `from A`。
 3. 接著呼叫父類別 `-Base()`，裡面執行 `method()`，在解構基底類別時虛擬機制會退化，故呼叫的是基底的 `method()`，印出 `from Base`。

HW2：Override (覆寫) 失敗的陷阱

- **題目內容：**解釋為何以下程式碼無法表現出多型？
`Base` 的 method 為 `virtual void foo() const` `Derived` 的 method 為 `void foo()`
- **輸出結果：**
`A's foo!A's foo!B's foo!`
- **細節解析 (老師重點)：**因為在 `Base` 裡面的 `foo()` 有宣告 `const`，但 `Derived` 裡面的沒有宣告 `const`，這導致兩者的 function signature 不一樣！因此這只是隱藏 (shadowing) 而沒有成功 override (覆寫) `base` 的 `foo`。即使基底有加上 `virtual`，編譯器仍視為兩個不同的 function，所以用 `Base*` 指標去呼叫時，依然只會輸出 `A's foo!`。

HW3：Java 的多型實作 (弦樂四重奏)

- **題目內容：**請以多型完成程式碼，演奏弦樂四重奏。
- **解析：**定義一個 abstract class `Instrument` 裡面有純虛擬方法 `abstract public void play();`。子類別 `Violin`, `Viola`, `Cello` 各自 `@Override` 了 `play()` 方法。
- **程式碼實作：**使用迴圈 `for(int i=0; i<4; i++){ stringQuartet[i].play(); }`，透過多型依次印出小提琴的「旋律」、中提琴的「合旋」與大提琴的「低音」。

HW4：實作 C++ 的 UML 繼承圖與多型

- **題目內容：**請使用多型實作符合 class diagram (`Base` 衍生出 `A` 與 `B`) 的程式碼，使輸出為 `Base`, `A`, `B`。
- **解析：**
建立 `vector<Base*> bases = { new Base(), new A(), new B() };`。
利用迴圈 `for (Base* b : bases) { b->print(); }` 進行多型呼叫。為了避免記憶體洩漏，必須再用一個迴圈將 `bases` 裡的指標 `delete` 並設為 `nullptr`，最後呼叫 `bases.clear();`。

HW5：C 語言如何實作多型 (使用 Function Pointer)

- **題目內容：**修改附件，使用 C 語言的機制模擬多型。
- **解析 (對應影片重點)：**如同老師影片中介紹在 OOP 發明前 C 語言的做法。在 `struct Book` 裡面宣告函數指標 `void (*print_type) (Book *self);`。在 `main` 函式中，動態將不同的函式位址 (`print_Comic`, `print_Novel`, `print_Magazine`) 綁定 (bind) 給該結構中的函數指標，達到執行時能印出不同書籍類型的動態連結效果。

HW6：實作純虛擬函式與 Abstract Class

- **題目內容：**寫一個 `main` 函式，利用多型讓動物發出聲音。輸出為 `meow` 及 `moo`。
- **解析：**
`Animal` 類別中定義了純虛擬函式 `virtual void speak() = 0;` (這使得 `Animal` 成為不可實體化的 Abstract Class)。
`Cat` 和 `Cow` 繼承自 `Animal` 並實作了 `speak()`。
主程式必須用基底類別指標建立子類別物件：`Animal *pet1 = new Cat; Animal *pet2 = new Cow;`，接著呼叫 `pet1->speak(); pet2->speak();`，最後不要忘記 `delete pet1; delete pet2;` 以歸還記憶體。